Concurrent programming

Εργασια 2

ΣΙΑΛΤΣΗΣ ΑΛΕΞΑΝΔΡΟΣ

ΜΑΡΗΣ ΑΘΑΝΑΣΙΟΣ

ΕΡΓΑΣΙΑ 2.1

- Η εργασία θέλει να υλοποιήσουμε τις συναρτήσεις για τους δυαδικούς σηματοφόρους βασιζόμενοι στους σηματοφόρους του System V.
- Οι συναρτήσεις που κληθήκαμε να υλοποιήσουμε είναι οι mysem_init, mysem_down, mysem_up, mysem_destroy

```
int mysem_init(mysem_t *s, int n) {
        if (n < 0 || n > 1) {
            return(0);
        }
        if (s->init != 0) {
            return(-1);
        }
        key = ftok(FILE_NAME,proj_id);
        proj_id++;
        s->semid = semget(key,1,0666 | IPC_CREAT);
        semctl(s->semid,0,SETVAL,n);
        s->init = 1;
        return(1);
}
```

```
int mysem_destroy(mysem_t *s) {
        if (s->init == 0) {
            return(-1);
        }
        semctl(s->semid,0,IPC_RMID);
        return(1);
}
```

```
Void mysem_calloc(mysem_t **s) {
        *s = calloc(1,sizeof(mysem_t));
}
```

```
Struct mysem_t {
    int semid;
    int init;
    struct sembuf op;
}
```

```
int mysem_down(mysem_t *s) {

        if (s->init == 0) {
            return(-1);
        }

        s->op.sem_num = 0;
        s->op.sem_op = -1;
        s->op.sem_flg = 0;

        semop(s->semid,&s->op,1);

        return(1);
}
```

```
int mysem_up(mysem_t *s) {

        if (s->init == 0) {
            return(-1);
        }

        s->op.sem_num = 0;
        s->op.sem_op = 1;
        s->op.sem_flg = 0;

        if (semctl(s->semid,0,GETVAL) == 0) {
            semop(s->semid,&s->op,1);
        }
        else {
            return(0);
        }

        return(1);


}
```

- Για να τεστάρουμε την υλοποίησή μας δημιουργήσαμε 3 threads τα οποία εκτυπώνουν ένα μήνυμα με πολλές εντολές printf για να δούμε ότι το ένα δεν διακόπτει το άλλο και ότι οι σηματοφόροι μας λειτουργούν σωστά

ΕΡΓΑΣΙΑ 2.2

- Η εργασία θέλει να αλλάξουμε το πρόγραμμα που αναπτύξαμε στην εργασία 1.2 , δηλαδή στο πρόγραμμα που βρίσκει αν οι αριθμοί που του δίνονται είναι πρώτοι ή όχι,χρησιμοποιώντας worker threads.

- Η υλοποίηση έπρεπε να γίνει με τους δυαδικούς σηματοφόρους που υλοποιήσαμε στην προηγούμενη άσκηση

Semaphores

- Κάθε worker που δημιουργείται έχει τον δικό του semaphore για να συγχρονίζεται με τη main
- Στην αρχή του loop μέσα στο thread του worker κάνει down έτσι ώστε να μπλοκάρει μέχρι να του δοθεί ο επόμενος αριθμός
- Όταν η main αναθέσει κάποιον αριθμό στον πρώτο available worker τότε κάνει up και ξεμπλοκάρει
- Οι semaphores αρχικοποιούνται στο 0 έτσι ώστε να μπλοκάρουν άμεσα

```
Void *prime (void *arg) {

        while(1) {
            down(s);

            n = *number;
            ans = 1;

            for (i = 2; i <= n/2; i++) {
                if (n % i == 0) {
                    ans = 0;
                    break;
                }
            }

            print(n : ans);

        }

        return(NULL);

}
```

```
int main() {

        for (i = 0; i < workers; i++) {
            mysem_init(s[i],0);
            pthread_create(prime,s[i]);
        }

        while(1) {
            scan(num);
            if (num == 0) break;

            find the next available worker(i)
            put num in i-th worker

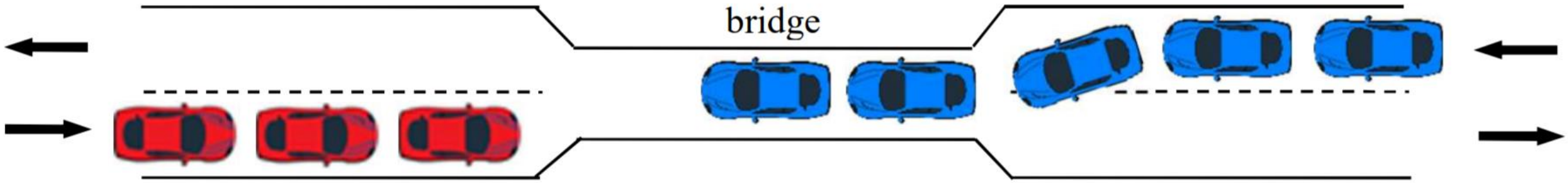
            up(s[i]);
        }

        blocking main until remaining workers are done

        return;
}
```

Εργασία 2.3

- Η εργασία θέλει να προσομοιάσουμε μια γέφυρα μονής κυκλοφορίας στην οποία δεν επιτρέπονται οχήματα που κινούνται και προς τις δύο κατευθύνσεις και να μην ξεπερνάνε ένα συγκεκριμένο αριθμό N.
- Το κάθε όχημα αποτελεί ένα thread και πρέπει τα threads να συγχρονιστούν μεταξύ τους χωρίς κάποιο βοηθητικό thread.
- Ο συχρονισμός έγινε με τους δικούς μας δυαδικούς σηματοφόρους



Semaphores

- Left_full,right full:Μπλοκάρουνε τα threads όταν η γέφυρα έχει το μέγιστο αριθμό οχημάτων
- Left_light,right_light: Μπλοκάρουνε τα threads όταν έρχονται οχήματα στη γέφυρα από την αντίθετη κατευθύνση
- Count_mtx:Προστατεύει τον κώδικα όταν αυξομειώνουμε τον αριθμό των οχημάτων πάνω στη γέφυρα
- Block_mtx:Προστατεύει τον κώδικα όταν ξεμπλοκάρουμε τα νήματα από τα left_full,right_full
- Enter_mtx:Προστατεύει τον κώδικα όταν κάνουμε αλλαγές πριν την είσοδο αμαξιού στη γέφυρα,όταν αυτή είναι άδεια

```
Int bridge_enter (int direction) {
        if (is_full == 1) {
            if (direction == FROM_LEFT) {
                down(left_full);
                down(block_mtx);
                woke_from_left++;
                if (woke_from_left < MAX_CARS) up(left_full);
                up(block_mtx);
            }
            else {
                down(right_full);
                down(block_mtx);
                woke_from_right++;
                if (woke_from_right < MAX_CARS) up(right_full);
                up(block_mtx);
            }
            down(block_mtx);
        }
        else {
            if (direction == FROM_LEFT) {
                down(block_mtx);
                woke_from_left++;
                up(block_mtx);
            }
            else {
                down(block_mtx);
                woke_from_right++;
                up(block_mtx);
            }
        }

        if(direction != bridge_direction && bridge direction != EMPTY)  {
            if (direction == FROM_LEFT) {
                down(left_light); up(left_light);
            }
            else {
                down(right_light); up(right_light);
            }
        }

        down(enter_mtx);
        if(car_count == 0) {
            bridge_direction = direction;
            if (direction == FROM_LEFT) down(right_light);
            else down(left_light);
        }
        down(count_mtx);car_count++;
        up(count_mtx);up(enter_mtx);
```

```
Int bridge_leave(int direction) {
        down(count_mtx);
        car_count--;
        up(count_mtx);
        down(block_mtx);
        if (direction == FROM_LEFT) woke_from_left--;
        else woke_from_right--;
        up(block_mtx);

        if (is_empty == 1) {
            bridge_direction = EMPTY;
            if (direction == FROM_LEFT) {
                up(left_full);
                if (woke_from_right < MAX_CARS) up(right_full);
                up(right_light);
            }
            else {
               up(right_full);
                if (woke_from_left < MAX_CARS) up(left_full);
                up(left_light);
            }
        }
}
```

Εργασία 2.4

- Η εργασία θέλει να προσομοιάσουμε ένα τρενάκι του λούνα παρκ και τους επιβάτες που θέλουν να ανέβουν.
- Το τρενάκι αποτελεί ένα thread και ο κάθε επιβάτης από ένα thread.
- Καλούμαστε να υλοποιήσουμε τον επιθυμητό συγχρονισμό ανάμεσα στο τρενάκι και τους επιβάτες αλλά και το συγχρονισμό μεταξύ των επιβατών
- Ο συχρονισμός έγινε με τους δικούς μας δυαδικούς σηματοφόρους

Semaphores

- Χρησιμοποιήσαμε 4 semaphores:
- Mtx: Κλειδώνει το κρίσιμο σημείο του train αλλά και του passenger(αρχικοποίηση στο 1)
- Mtx2:Κλειδώνει τα κρίσιμα σημεία των passengers μεταξύ τους(αρχικοποίηση στο 1)
- Passengers:Μπλοκάρει τα νήματα των επιβατών όταν αυτά πρέπει να περιμένουν(αρχικοποίηση στο 1)
- Train:Μπλοκάρει το νήμα του τρένου μέχρι το τρένο να γεμίσει(αρχικοποίηση στο 0 έτσι ώστε κατά την εκκίνηση να μπλοκάρει)

```
Void *train_thread(void *arg) {
        while(1) {

            //train is going to wait

            down(train);

            down(mtx);

            //ride is full and the train begins
            // riding the train……..
            //the ride ended

            avl = capacity;
            up(passengers);
            up(mtx);

        }
        return(NULL);
}
```

```
Void *passenger_thread(void *arg) {
        down(mtx2);
        if (avl == 0) {
            down(passengers);
            up(passengers);
        }
        down(mtx);
        if (avl > 0) {
            avl--;
            //passengers on board
        }
        if (avl == 0) {
            //waking up the train
            down(passengers);
            up(train);
        }
        up(mtx2);
        up(mtx);
        return(NULL);
}
```